

Resolution Where It Counts: Hash-based GPU-Accelerated 3D Reconstruction via Variance-Adaptive Voxel Grids

LORENZO DE REBOTTI, Sapienza, University of Rome, Italy
 EMANUELE GIACOMINI, Sapienza, University of Rome, Italy
 GIORGIO GRISETTI, Sapienza, University of Rome, Italy
 LUCA DI GIAMMARINO, Sapienza, University of Rome, Italy

<https://rvp-group.github.io/mrhash/>



Fig. 1. **Overview.** Our pipeline for real-time 3D reconstruction and rendering. *Left:* Our variance-adaptive voxel grid dynamically adjusts resolution based on TSDF variance, enabling compact and accurate reconstructions from both RGB-D and unstructured 3D point clouds. The mesh output supports high-quality rendering via Gaussian Splatting, with adaptive splat control handled fully on the GPU. *Right:* Overview comparison reporting F-score across different datasets, memory efficiency, and FPS.

Efficient and scalable 3D surface reconstruction from range data remains a core challenge in computer graphics and vision, particularly in real-time and resource-constrained scenarios. Traditional volumetric methods based on fixed-resolution voxel grids or hierarchical structures like octrees often suffer from memory inefficiency, computational overhead, and a lack of GPU support. We propose a novel variance-adaptive, multi-resolution voxel grid that dynamically adjusts voxel size based on the local variance of signed distance field (SDF) observations. Unlike prior multi-resolution approaches that rely on recursive octree structures, our method leverages a flat spatial hash table to store all voxel blocks, supporting constant-time access and full GPU parallelism. This design enables high memory efficiency and real-time scalability. We further demonstrate how our representation supports GPU-accelerated rendering through a parallel quad-tree structure for Gaussian Splatting, enabling effective control over splat density. Our open-source CUDA/C++ implementation achieves up to 13× speedup and 4×

lower memory usage compared to fixed-resolution baselines, while maintaining on par results in terms of reconstruction accuracy, offering a practical and extensible solution for high-performance 3D reconstruction.

CCS Concepts: • **Computing methodologies** → **Volumetric models; Mesh models; Rendering; Perception.**

Additional Key Words and Phrases: Surface Reconstruction, Novel View Synthesis, Gaussian Splatting

ACM Reference Format:

Lorenzo De Rebotti, Emanuele Giacomini, Giorgio Grisetti, and Luca Di Giammarino. 2025. Resolution Where It Counts: Hash-based GPU-Accelerated 3D Reconstruction via Variance-Adaptive Voxel Grids. *ACM Trans. Graph.* 1, 1 (November 2025), 16 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Accurate and scalable 3D reconstruction [Di Giammarino et al. 2023; Giacomini et al. 2025] from depth or point cloud data is a core challenge in computer graphics, robotics, and vision. Applications such as robotic navigation, AR/VR, and large-scale mapping demand representations that are both geometrically precise and computationally efficient. Volumetric methods, particularly the Truncated Signed Distance Function (TSDF), are widely used for this purpose. Introduced by Curless and Levoy [Curless and Levoy 1996], TSDF-based fusion integrates depth over time into a voxel grid, enabling robust surface reconstruction. Traditional approaches [Newcombe et al. 2011; Nießner et al. 2013; Oleynikova et al. 2017] employ uniform grids, which scale poorly due to high memory and compute demands.

Authors' Contact Information: Lorenzo De Rebotti, Sapienza, University of Rome, Rome, Italy, derebotti@diag.uniroma1.it; Emanuele Giacomini, Sapienza, University of Rome, Rome, Italy, giacomini@diag.uniroma1.it; Giorgio Grisetti, Sapienza, University of Rome, Rome, Italy, grisetti@diag.uniroma1.it; Luca Di Giammarino, Sapienza, University of Rome, Rome, Italy, digiammarino@diag.uniroma1.it.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
 ACM 1557-7368/2025/11-ART
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

To improve efficiency, adaptive structures like octrees and OpenVDB [Museth 2013] have been proposed. Octrees hierarchically allocate memory where needed but suffer from recursive traversal and poor GPU performance. OpenVDB provides sparse multi-resolution storage through a tree structure, and variants such as VDBFusion [Vizzo et al. 2022] have extended it to TSDF fusion. Despite their versatility, these structures introduce non-trivial access overhead and are not optimized for real-time, GPU-based reconstruction. Furthermore, many existing multi-resolution approaches depend on input-specific cues (e.g., semantics or depth confidence), limiting their generality and adaptability across sensor types.

In this work, we propose *MrHash*, a novel variance-adaptive, multi-resolution voxel grid implemented entirely on the GPU using a flat spatial hash table. Our system adjusts resolution based on local TSDF variance, independent of sensor modality or semantic priors, enabling fine detail where needed and coarse representation elsewhere. Unlike octrees, our approach avoids hierarchical traversal, achieving, on average, constant-time access and real-time performance.

Our main contributions are:

- A variance-adaptive voxel grid that adjusts resolution based on local TSDF variance, preserving fine detail and reducing memory in uniform areas. We extend Marching Cubes to support seamless meshing across resolution boundaries.
- A flat hash table structure that manages mixed-resolution voxel blocks without hierarchical overhead, enabling efficient GPU integration and access.
- A fully parallel GPU-based quad-tree for controlling the spatial distribution of Gaussians for novel view synthesis, allowing adaptive splat density without CPU-GPU streaming.
- An open-source CUDA/C++ implementation supporting real-time reconstruction and rendering, designed for extensibility and community use.

2 Related Work

3D mapping systems typically represent geometry as either explicit sparse maps [Brizi et al. 2024; Ćwian et al. 2025] or implicit volumetric fields [Nießner et al. 2013]. Explicit pipelines maintain sparse local submaps composed of samples or parametric primitives (e.g., point clouds, surfels, or 2D/3D Gaussian splats) that are well-suited for scan-to-scan [Di Giammarino et al. 2022] or scan-to-submap registration and odometry [Ferrari et al. 2024; Schöps et al. 2019; Zhang and Singh 2014]. In contrast, implicit volumetric approaches store a discretized signed distance field, enabling robust multi-view fusion, denoising, and direct meshing at the cost of higher memory and compute.

Volumetric 3D reconstruction using the TSDF has been a foundational approach for over three decades. The seminal work by Curless and Levoy [Curless and Levoy 1996] introduced volumetric fusion of depth maps, enabling robust surface reconstruction through SDF accumulation. Although initially restricted to offline processing, this approach laid the groundwork for real-time systems. KinectFusion [Newcombe et al. 2011] demonstrated real-time TSDF integration on a dense, uniform voxel grid, enabling accurate reconstructions in small, bounded scenes. However, the fixed resolution

and exhaustive memory allocation made the approach unsuitable for large-scale environments. Voxel Hashing [Nießner et al. 2013] addressed this limitation by dynamically allocating voxel blocks using a spatial hash table, reducing memory usage and enabling real-time mapping for mobile platforms.

Building on this foundation, works like Voxblox [Oleynikova et al. 2017] and Supereight [Vespa et al. 2019] further improved scalability. Voxblox uses block-wise hashing and ray bundling to support onboard updates on resource-constrained UAVs, while Supereight introduces an adaptive allocation strategy to improve runtime and memory efficiency. These systems prioritize real-time performance, often at the cost of geometric precision.

OpenVDB [Museth 2013] and the newer deep learning framework [Williams et al. 2024] offer a more general-purpose sparse volumetric data structure widely used in graphics and VFX. OpenVDB supports multi-resolution storage through a hierarchical tree structure and has been adapted to 3D reconstruction in works like VDBFusion [Vizzo et al. 2022], which applies it to unstructured 3D point cloud fusion. However, OpenVDB’s multi-level hierarchy introduces non-trivial lookup costs and is not well-suited for real-time integration. Additionally, each leaf node operates at a fixed resolution, limiting local adaptivity within a region.

In parallel, learning-based representations have emerged that model 3D geometry via continuous neural fields. Methods such as InstantNGP [Müller et al. 2022], NICER-SLAM [Zhu et al. 2024], and PIN-SLAM [Pan et al. 2024] leverage hierarchical feature grids and MLP decoding to reconstruct high-quality geometry while incorporating SLAM constraints. Although these approaches achieve accurate results, they are typically limited to small-scale scenes and require substantial computational and memory resources. Fully neural pipelines such as NKSR [Huang et al. 2023] and N³-Mapping [Song et al. 2024] achieve high fidelity even under sparse observations; however, both depend on computationally intensive per-scene neural optimization and demand high GPU resources, often leading to failures when scaling to large-scale scenarios.

Recent works have explored adaptive resolution guided by semantics or image gradients [Zheng et al. 2024], enabling selective detail refinement. However, such approaches depend on structured RGB-D input and do not generalize to unstructured 3D point clouds. Similarly, Funk et al. [Funk et al. 2021] use octree structures for adaptive occupancy mapping, but rely on log-odds surface modeling and suffer from hierarchical access overhead.

More recently, 3D Gaussian Splatting [Kerbl et al. 2023] has been combined with TSDF mapping, as in GSFusion [Wei and Leutenegger 2024]. They extend Supereight2 [Funk et al. 2021] to support online rendering. The TSDF layer provides stable geometry for constraining Gaussians initialization and optimization.

Unlike prior work, we introduce a variance-driven, multi-resolution voxel grid that is sensor-agnostic and GPU-native. Instead of relying on semantic cues or hierarchical traversal, our system adaptively allocates voxel resolution based on local TSDF variance, using a flat hash table. This enables constant-time access, resolution mixing, and real-time performance across both RGB-D and LiDAR inputs, without sacrificing geometric fidelity or memory efficiency.

3 Technical Section

The TSDF map representation used consists of a collection of spatially hashed voxels V_i with a variable voxel size $v \in \mathbb{R}^+$, which is determined based on the density of neighboring voxels. Voxels of the same size are stored in B_j blocks (cuboids), each of which contains a number of voxels that varies with the voxel resolution. Each block B_j is identified by a global position $\mathbf{b}_j = (x, y, z)^T \in \mathbb{Z}^3$. A voxel V_i mainly stores a truncated signed distance $D_i \in \mathbb{R}$, a weight $W_i \in \mathbb{N}$ that reflects the confidence in D_i , the color, and the variance of the mean TSDF σ_i^2 .

At each frame k , the system integrates the current sensor's measurement (either a raw point cloud or a dense image).

3.1 Integration

We integrate 3D sensor data into a TSDF volume using a standard volumetric fusion strategy [Curless and Levoy 1996]. The TSDF provides an implicit surface representation, where each voxel V_i encodes a scalar value D_i indicating the signed distance from the voxel center to the closest observed surface along the sensor's line of sight. Positive values represent free space (outside the surface), while negative values correspond to space behind the surface (inside objects). To restrict updates to relevant regions, the distance is truncated beyond a fixed threshold $\tau > 0$. Depending on the type of measurement and the availability of dense color data, we distinguish between different integration logic.

3.1.1 3D Point Cloud. For 3D point cloud measurements (i.e., LiDAR sensors), we perform ray-based geometric integration using a DDA traversal [Amanatides and Woo 1987] from the sensor origin to each point $\mathbf{p}$. Voxels intersected by the ray are updated based on their signed distance to $\mathbf{p}$. For a voxel center at position $\mathbf{x}$ and a surface point $\mathbf{p}$ observed by the sensor, the truncated signed distance is defined as:

$$d_k(\mathbf{x}) = \text{clip}((\mathbf{p} - \mathbf{x}) \cdot \hat{\mathbf{n}}, -\tau, \tau), \quad (1)$$

where $\hat{\mathbf{n}} = \frac{\mathbf{p} - \mathbf{o}}{\|\mathbf{p} - \mathbf{o}\|}$ is the unit ray direction from the sensor origin $\mathbf{o}$ to the point $\mathbf{p}$.

3.1.2 Cameras. Differently, for cameras with providing depth information (i.e., RGB-D), we use a projective mapping approach. Each voxel center $\mathbf{x}$ is projected onto the image plane to obtain pixel coordinates (u, v) . The corresponding depth value d at that pixel defines the measured distance along the viewing ray. The signed distance is then computed as the difference between the measured depth and the voxel depth $\|\mathbf{x}\|$ along the same ray:

$$d_k(\mathbf{x}) = \text{clip}(d - \|\mathbf{x}\|, -\tau, \tau). \quad (2)$$

We note that the above integration strategy follows standard volumetric fusion techniques [Curless and Levoy 1996; Nießner et al. 2013], and is not a core contribution of our work. However, we include it here for completeness and clarity of exposition.

Independent of the integration type, each voxel maintains a weighted average of these observations; the distance D_i and the weight W_i are updated as follows:

$$D_i \leftarrow \frac{W_i D_i + w_k d_k(\mathbf{x})}{W_i + w_k}, \quad W_i \leftarrow W_i + w_k. \quad (3)$$

Here, w_k is a confidence weight. While it is often set based on sensor noise or distance [Curless and Levoy 1996; Nießner et al. 2013], we fix $w_k = 1$ to simplify the computation of voxel-wise variance. Different from other works, with the running average D_i , we maintain the sample variance σ_i^2 of the TSDF values using an online update. This variance reflects the local geometric complexity: low in flat regions, high near surface boundaries, or noise. We use σ_i^2 to guide voxel resolution, preserving fine detail where needed and merging voxels in smoother areas, enabling adaptive resolution and improved memory efficiency.

3.2 Grid Representation

Given the inherent sparsity of most 3D environments, a dense volumetric grid would be highly impractical for large-scale scenarios. Instead, we adopt a sparse volumetric representation based on a hash table, as introduced in [Nießner et al. 2013], which enables memory-efficient storage and access to active voxel blocks. While this approach introduces additional challenges related to data locality and memory coherence, it allows for efficient large-scale scene reconstruction without requiring predefined spatial bounds. One of our main contributions is a multi-resolution grid tailored for a single hash table. Unlike fixed-voxel-size methods [Nießner et al. 2013; Oleynikova et al. 2017], we allocate voxel blocks of a constant metric size, while varying their internal resolution by adjusting the number of voxels contained within each block. Each entry in the hash table H contains:

- $\mathbf{b}_j \in \mathbb{Z}^3$: integer coordinates of the voxel block origin (lower-left corner), used as the spatial key;
- $\mathbf{o}_j \in \mathbb{N}$: auxiliary offset for resolving hash collisions;
- $t_j \in \mathbb{Z}$: pointer to the voxel block data stored in GPU memory;
- $n_j \in \mathbb{N}$: index of the resolution level corresponding to heap $\mathbf{h}_n$.

Each hash entry uniquely identifies a voxel block B_j through its spatial key $\mathbf{b}_j$, which is mapped to an index by the hash function H , defined as:

$$H(x, y, z) = (x \cdot p_1 \oplus y \cdot p_2 \oplus z \cdot p_3) \bmod n_{\text{hash}} \quad (4)$$

where $p_1 = 73856093$, $p_2 = 19349669$, $p_3 = 83492791$ are large prime numbers [Teschner et al. 2003] and n_{hash} is the number of entries in the hash table.

As in [Nießner et al. 2013], to handle collisions, each entry in the hash table maintains a fixed-size bucket for up to N voxel blocks. In the rare event of overflow, additional blocks are appended to a per-entry linked list. In addition, as is commonly done [Nießner et al. 2013; Oleynikova et al. 2017] to allocate all intersected voxel blocks efficiently, we leverage DDA [Amanatides and Woo 1987].

The key idea of our method is having a unified hash table supporting a multi-resolution voxel representation within a single address space. Each entry of the hash table stores a pointer to the voxel block allocated in heap $\mathbf{h}_n$ associated with that resolution level. This structure allows voxel blocks of different resolutions to be indexed and accessed through the same hash table (see Fig. 3).

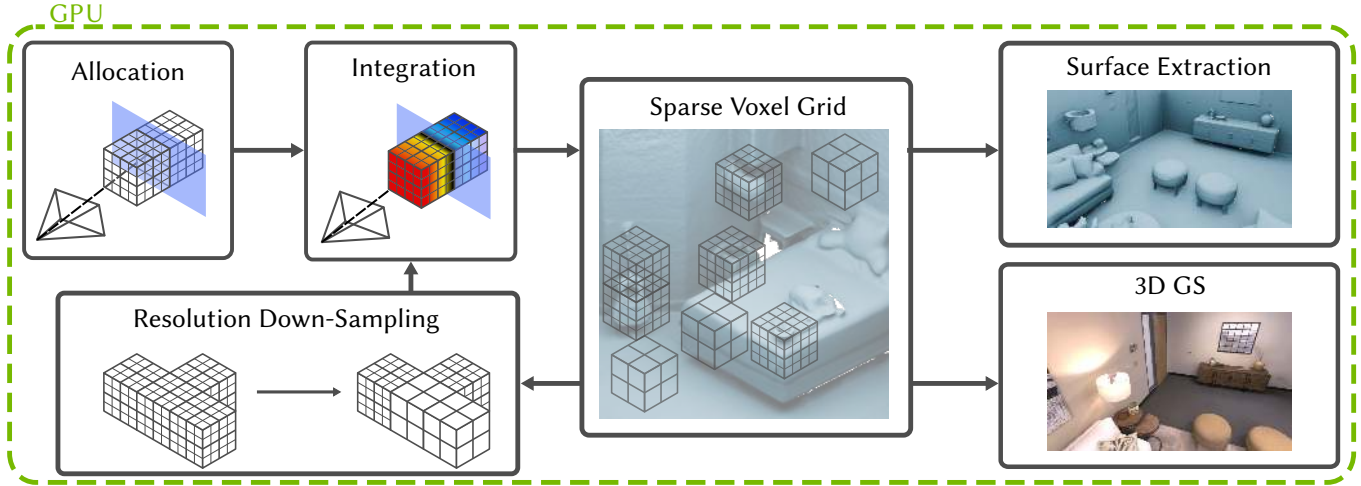


Fig. 2. **System Overview:** *MrHash* takes the input data and, if necessary, allocates the voxel blocks associated with such 3D points and integrates them into a multi-resolution sparse voxel grid. The blocks whose variance is below a predefined threshold are downsampled and re-integrated into the voxel grid at a coarser resolution. Moreover, our pipeline can either extract the isosurface from the volumetric representation or render the surrounding environment.

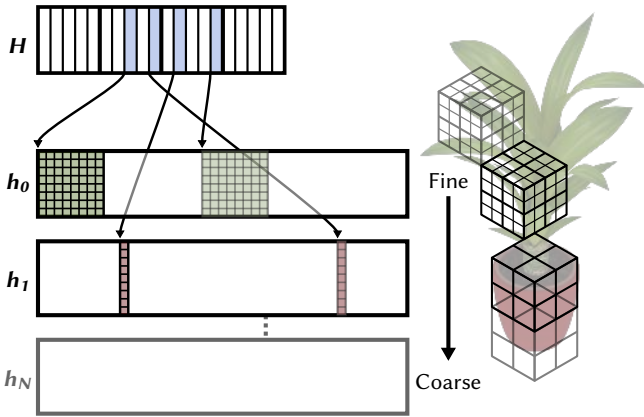


Fig. 3. **Illustration of grid representation to hash table mapping.** The hash function maps the world points from integer world coordinates to the respective buckets of the hash-table H . Every entry of the hash-table contains a pointer to a block in the specific heap of that resolution.

While managing multiple resolutions within a flat hash table increases implementation complexity (e.g., in hashing, memory layout, and consistency), it enables a more flexible and efficient system compared to traditional hierarchical representations [Vespa et al. 2018]. Most multi-resolution volumetric approaches use octree hierarchies [Funk et al. 2021], which naturally support dyadic spatial subdivision and adaptive refinement near surfaces. However, octrees incur higher computational cost, typically $O(\log N)$, for insertion, deletion, and neighborhood queries due to recursive traversal and pointer-based memory layouts. Even methods leveraging hybrid tree-based data structures, such as those proposed in [Museth 2013, 2021], provide highly efficient sparse volumetric representations; however, their hierarchical organization introduces considerable

overhead for frequent updates and random access. In contrast, our method achieves constant-time access $O(1)$ on average for these operations by encoding multi-resolution voxel blocks directly into a flat hash table.

Resolution is handled explicitly through key encoding, enabling the simultaneous representation of voxels at different scales within a unified, non-hierarchical structure. This avoids the overhead of recursive descent and facilitates more efficient parallel execution on modern hardware, particularly in GPU-based implementations where memory indirection and dynamic branching are costly. We show the benefits of our implementation in Sec. 4.

3.2.1 Voxel Merging Criterion. We initialize the voxel blocks to the finest resolution, since “ideally” this provides the best geometrical accuracy. Then, to preserve surface complexity and handle high-frequency signals, we maintain higher voxel resolution in areas with high TSDF variance over time. Instead, for regions exhibiting low variance, we merge voxels of the same block, as in Fig. 4.

Driven exclusively by TSDF statistics, the approach is sensor-agnostic, requiring no sensor-specific assumptions or noise parameters. We compute variance using a numerically stable, single-pass method based on Welford’s algorithm [Welford 1962].

Given a sequence of k observations $d_1, d_2, \dots, d_k$ for a voxel V_i , we maintain the running mean, as defined in Eq. (3), and the accumulated sum of squared deviations $S_{2,i,k}$, updated as:

$$S_{2,i,k} = S_{2,i,k-1} + (d_k - D_{i,k-1})(d_k - D_{i,k}), \quad (5)$$

from which the variance σ_i^2 is computed as:

$$\sigma_i^2 = \frac{S_{2,i,k}}{k}. \quad (6)$$

This formulation enables incremental computation of voxel-wise variance without storing the full observation history. At each integration step, variance is updated for all voxels and aggregated at

the block level to obtain the mean variance of each block. Because this operation is computationally demanding, we implement it in a fully parallel manner on the GPU, using intra-block reductions to efficiently accumulate the $S_{2,i,k}$ terms across voxels.

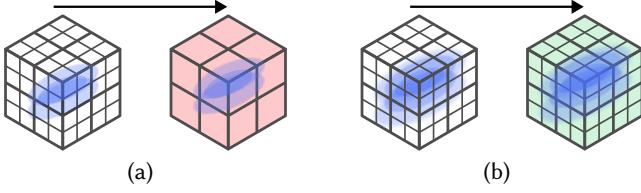


Fig. 4. **Adopted merging strategy.** a) Voxel block with low TSDF variance is reallocated at a coarser resolution (red). b) Voxel block with high TSDF variance remains unchanged (green).

3.3 Streaming

GPU memory capacity and bandwidth are fundamental bottlenecks in large-scale 3D reconstruction, where volumetric data scales with the environment’s size. Retaining all voxel data in GPU memory becomes infeasible for expansive scenes, and frequent CPU-GPU transfers introduce non-negligible latency due to limited bandwidth and synchronization costs. To address this, it is essential to adopt a streaming strategy that efficiently manages data movement.

Unlike previous systems such as [Nießner et al. 2013], which continuously stream voxel blocks based on geometric heuristics (e.g., frustum culling or proximity thresholds), our method defers streaming until GPU memory usage approaches a user-defined capacity threshold (e.g., 85% of the total GPU memory allocated for voxel data). Once this limit is reached, blocks are selected for removal from the GPU memory based on spatial relevance: for RGB-D input, we consider blocks outside the current camera frustum; for LiDAR input, we consider blocks lying beyond a predefined radius from the current sensor pose. This deferred, capacity-driven policy minimizes unnecessary data transfers and synchronization overhead, maintaining GPU resources for actively reconstructed regions.

3.4 Multi-resolution Marching Cubes

To extract the isosurface from the TSDF, we implement the Marching Cubes algorithm [Lorensen and Cline 1987] directly on the multi-resolution voxel grid, leveraging GPU parallelism. Each voxel is processed independently, using local signed distance values that are tri-linearly interpolated at its corners. Although voxel size does not affect the core algorithm, resolution boundaries pose a challenge: interpolating corner values involves querying neighboring voxels, whose TSDF values are defined relative to different centers (see Fig. 5). Zheng et al. [Zheng et al. 2024] address this by uniformly refining neighboring voxels, which increases memory usage in regions that would otherwise remain coarse. Instead, we assume limited variation at the scales considered and interpolate between coarse and fine voxels using a weighted scheme that favors the finer resolution where available. Unlike voxel-based terrain rendering in computer graphics, such as Transvoxel [Lengyel 2010], a volumetric integration system like ours does not allow arbitrary allocation of

voxel blocks, as their creation is driven by incoming measurements and does not account for neighboring voxel resolutions. Hence, outer coarse voxels may spatially overlap with adjacent finer-resolution voxels. To resolve this, inspired by [Lengyel 2010], we truncate coarse voxels along the dimensions corresponding to faces shared with finer neighbors, ensuring a clean partitioning of space and preventing multiple faces from being generated within the same spatial region (see Fig. 6). To further mitigate artifacts from these approximations, we apply a vertex collapsing step (based on voxel size) during mesh generation, simplifying redundant geometry and improving consistency across resolution transitions.

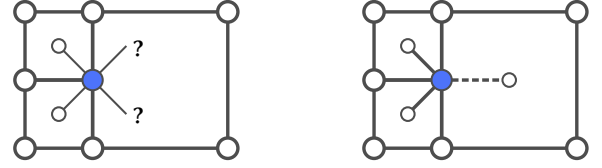


Fig. 5. **Ambiguity of Signed Distance Function (SDF) interpolation.** The blue vertex attempts to interpolate the Signed Distance Function (SDF) value from its four neighboring voxels during sampling, but encounters undefined entries due to missing neighbors. To address this, we apply a weighted interpolation scheme that prioritizes finer-resolution values where available. For clarity, the visualization is shown in 2D assuming bilinear interpolation.

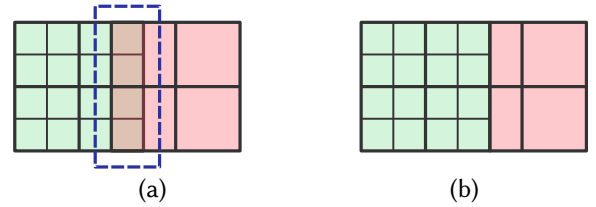


Fig. 6. **Transitional voxels.** (a) Due to the measurement-driven allocation, overlapping regions (dotted area) can occur between voxel blocks of different resolutions, which poses challenges during the vertex evaluation step in Marching Cubes. (b) To address this, we reduce the size of coarser voxels and adjust the placement of Marching Cubes vertices accordingly to maintain consistency across resolutions.

3.5 Rendering

We employ 3D Gaussian Splatting [Kerbl et al. 2023] for the Novel View Synthesis (NVS) task. In this representation, the scene is modeled as a collection of M 3D Gaussian primitives, each parametrized by a mean position $\mu_{m,w}$ and a covariance matrix $\Sigma_{m,w}$, both defined in world coordinates, along with a view-dependent color C_m , and an opacity α_m .

Each 3D Gaussian in world coordinates is projected onto the image plane via a projective transformation π :

$$\mu_{m,l} = \pi(T_{CW}\mu_{m,w}), \quad \Sigma_{m,l} = J\mathbf{R}\Sigma_{m,w}\mathbf{R}^T J^T \quad (7)$$

where $\mathbf{T}_{CW} \in \mathbb{SE}(3)$ is the world-to-camera transform matrix, $\mathbf{J}$ is the Jacobian of the projective transformation π , and $\mathbf{R} \in \mathbb{SO}(3)$ is the rotation component of $\mathbf{T}_{CW}$. Since the covariance is defined in image space, the resulting $\Sigma_{m,l}$ is obtained by taking the upper-left 2×2 submatrix of the projected 3×3 covariance. This formulation is fully differentiable; the means, covariances, colors, and opacities of the Gaussian can be jointly optimized from a set of posed input images using Stochastic Gradient Descent. The optimization process minimizes the photometric difference between the reference views and the corresponding rendered images. These are generated by a dedicated tile-based rasterizer that performs efficient α -blending composing of the M primitives in screen space. For each pixel (u, v) , the final color is computed as:

$$C_p(u, v) = \sum_{m=1}^M C_m \alpha_m \mathcal{G}_{m,l}(u, v) \prod_{l=1}^{m-1} (1 - \alpha_l \mathcal{G}_{l,l}(u, v)) \quad (8)$$

where the primitives are indexed by depth, from front to back, and $\mathcal{G}_{m,l}(u, v)$ is the evaluation of the projected m -th Gaussian at the corresponding pixel, computed as

$$\mathcal{G}_{m,l}(u, v) = \exp \left(-\frac{1}{2} ((u, v) - \mu_{m,l})^T \Sigma_{m,l}^{-1} ((u, v) - \mu_{m,l}) \right) \quad (9)$$

During optimization, the number of Gaussians is managed by an adaptive control mechanism that, based on gradient information, either prunes or densifies the primitives within specific regions. Adaptive control of Gaussian density is key to an efficient representation, and the original implementation has several critical flaws. First, some areas may be oversaturated by primitives, increasing the rendering times, memory consumption, and training time. Moreover, densification stems from existing Gaussians, so initial seeding is crucial to ensure fast convergence.

Following the new trend in 3DGS-based SLAM [Keetha et al. 2024; Matsuki et al. 2024; Peng et al. 2024], we reformulate 3DGS in an incremental manner, processing temporally-ordered posed images sequentially.

The Gaussians are seeded using depth information, effectively solving the second problem, while our underlying voxel grid enables a more structured, efficient approach to controlling Gaussians and addresses oversaturation. To regulate Gaussian density, we construct an image-space quadtree guided by visibility and density cues derived from the voxel grid. Each node represents an image region whose contrast, i.e., the luminance-weighted RGB mean-squared error, quantifies local intensity variation and determines whether further subdivision is required.

To estimate the contrast c of a quad-tree Q , we consider the RGB values $\mathbf{q} \in \mathbb{R}^3$ for every pixel of the quad-tree and compute the following:

$$c(Q) = 1 \frac{\sum_{\mathbf{q} \in Q} (\mathbf{q} - \text{mean}_{\text{rgb}}(Q))^2}{|Q|} \quad (10)$$

where $\mathbf{1} = [0.2989, 0.5870, 0.1140]$ are the Rec. 601 luma weights, $\text{mean}_{\text{rgb}}(Q) \in \mathbb{R}^3$ are the mean values for the three channels of the quadtree's pixels, and $|Q|$ is the number of pixels of the quadtree. Within this context, $(\mathbf{q} - \text{mean}_{\text{rgb}}(Q))^2$ denotes the element-wise square of the difference vector.

Similar to [Wei and Leutenegger 2024], the quadtrees are constructed by this contrast-based approach, dividing the image so that each leaf node contains a subregion with contrast below a threshold or a minimum pixel size. Unlike them, we construct quadtrees on the GPU and implement the operations in Eq. (10) using the sum-reduction technique.

To maintain high throughput, both contrast estimation and quadtree construction are executed in parallel using a breadth-first strategy, avoiding recursion and leveraging shared memory for reductions (see Alg. 1). Therefore, all supporting data already resides in GPU memory, eliminating costly transfers and accelerating the auxiliary computations that guide adaptive splat generation.

ALGORITHM 1: GPU-Based Quadtree Subdivision (Parallel Version)

Data : Input image, contrast threshold, minimum pixel size
Result : Final set of quadtree leaves
Initialize *RootNode* covering the entire image;
NodeQueue $\leftarrow$ {*RootNode*};
while *NodeQueue* $\neq$ $\emptyset$ **do**
 foreach *Node* $\in$ *NodeQueue* **do** // executed in parallel
 | *Node*.Contrast $\leftarrow$ ComputeContrastWSharedMem (*Node*,
 | Image);
 end
 NewQueue $\leftarrow$ $\emptyset$;
 foreach *Node* $\in$ *NodeQueue* **do** // executed in parallel
 | **if** *Node*.Contrast > Threshold **and**
 | *Node*.Size > MinPixelSize **then**
 | *Children* $\leftarrow$ Subdivide (*Node*);
 | add *Children* to *NewQueue*;
 | **end**
 | **else**
 | add *Node* to *FinalLeaves*;
 | **end**
 end
 NodeQueue $\leftarrow$ *NewQueue*;
end
return *FinalLeaves*;

Procedure SUBDIVIDE(*Node*)
 Compute midpoints in x and y ;
 Create 4 children: TopLeft, BottomLeft, TopRight, BottomRight;
 return *Children*;

4 Experiments

In this section, we report the results of our method on several publicly available datasets. We evaluate *MrHash* for both mapping accuracy and rendering performance. To the best of our knowledge, ours is the first approach to combine a variance-driven, multi-resolution voxel grid with a single flat hash table, enabling fully GPU-based TSDF integration and rendering.

4.1 Experimental Settings

4.1.1 Datasets. To evaluate the performance of our method, we used several publicly available benchmarks that feature heterogeneous sequences and varying resolutions across different sensor

types. RGB-D Replica Dataset [Straub et al. 2019] is a synthetic benchmark consisting of 18 photorealistic, semantically annotated indoor environments at room scale. Each scene provides dense meshes textured with high-resolution HDR imagery, per-primitive semantic class and instance annotations, as well as explicit modeling of reflective surfaces such as glass and mirrors. RGB-D images are rendered from ground-truth poses using the official renderer. Scannet [Dai et al. 2017a] was constructed from 2.5M RGB-D frames across 1513 indoor scenes, recorded using iPad-mounted depth sensors. The raw video streams were processed with BundleFusion [Dai et al. 2017b] to yield globally consistent meshes, which were then densely annotated via large-scale crowdsourcing into semantic and instance-level segmentations. Ground truth includes camera intrinsics, 6-DoF poses, and reconstructed surfaces. Newer College Dataset (NC) [Zhang et al. 2021] was collected using a handheld Ouster OS0-128 LiDAR across both structured environments and vegetated areas. Ground truth was obtained with a Leica BLK360 scanner, providing centimeter-level accuracy for both poses and map points. Oxford Spires Dataset [Tao et al. 2025] was recorded using a handheld Hesai QT64 LiDAR sensor, which provides a 360° horizontal field of view, 104° vertical field of view, 64 vertical channels, and a maximum range of 60 meters. Each sequence is accompanied by a survey-grade 3D laser scan, serving as a prior map for ground-truth trajectory estimation and mapping evaluation.

4.1.2 Evaluation Metrics. To evaluate mapping quality, we adopt standard surface reconstruction metrics as defined in [Mescheder et al. 2019]. Accuracy (Acc) measures the average distance from points on the reconstructed mesh to their nearest neighbors in the reference point cloud, while Completeness (Comp) captures the inverse, how well the reconstruction covers the reference surface. The Chamfer- L_1 distance ($C-L_1$) is the mean of these two and provides a balanced measure of geometric fidelity. Additionally, we compute the F-score with a 10 cm error threshold for the RGB-D and 20 cm for the LiDAR. F-score evaluates geometric consistency by combining precision and recall into a single value. The rendering quality is measured comparing Peak Signal-to-Noise Ratio (PSNR), Structure Similarity Index Measure (SSIM) [Wang et al. 2004], and Learned Perceptual Image Patch Similarity (LPIPS) [Zhang et al. 2018].

4.1.3 Baselines. We compared our method against popular mapping SOTA pipelines. We include VDBFusion [Vizzo et al. 2022], VoxBlox [Oleynikova et al. 2017], and Supereight2 [Funk et al. 2021] as purely geometric baselines. VDBFusion employs OpenVDB [Museth 2013] as a volumetric data structure to handle 3D points, VoxBlox combines adaptive weights and grouped ray-casting for an efficient TSDF integration, while Supereight2 supports a multi-resolution grid through an octree implementation. Additionally, we include two neural-implicit mapping pipelines. N^3 -Mapping [Song et al. 2024] is a neural-implicit non-projective Signed Distance Function (SDF) mapping pipeline. It leverages normal guidance to produce more accurate SDFs, leading to SOTA results for offline LiDAR mapping. PIN-SLAM [Pan et al. 2024], leverages neural points as primitives and interleaves an incremental learning of the model’s SDF and, using marching cubes, it can produce a mesh from the underlying implicit SDF. In addition, we compare our method with other approaches that incorporated Gaussian Splatting into their

map representation, like [Wei and Leutenegger 2024]. Different from us, they employed an octree-based map representation, which aids the splats initialization. For both 3D reconstruction and rendering experiments, we ran all the pipelines with ground-truth poses.

4.1.4 Implementation. The experiments were run on a PC with an Intel Core i9-13900K @ 3.20Ghz, 128GB of RAM, and an NVIDIA RTX 4090 GPU with 24 GB of VRAM. For all experiments where we employed our single-resolution configuration, we used a block size of 512 voxels. Also, for the finer resolution of the multi-resolution grid, we use a block size of 512; consequently, for the coarser resolution, there are 64 blocks. For the hash table, we employed a bucket size of 10 hash entries, an eventual linked list size of 7 elements, and, as presented in Sec. 3.2, the values used to compute the hash of the block coordinates are: $p_1 = 73856093$, $p_2 = 19349669$, $p_3 = 83492791$. In the rendering experiments, all the approaches used the same parameters to subdivide the image into quadrees: a threshold contrast of 0.1 and a minimum pixel size of 1. To optimize the Gaussian parameters, we used the same solver configuration for all methods, except for Ours (+iterations), where we applied our single-resolution grid and increased the number of iterations in the Gaussian Splatting solver.

Our results show that our method (supporting both RGB-D and point cloud inputs) offers improved performance and efficiency while maintaining a SOTA accuracy in 3D reconstruction.

4.2 3D Reconstruction

Our method achieves reconstruction quality comparable to or better than prior approaches across multiple datasets. The results demonstrate that our variance-adaptive multi-resolution grid does not compromise accuracy, even while significantly reducing memory usage (Tab. 3). Notably, N^3 -Mapping failed to complete any sequence from the NC dataset, likely due to its high computational and memory requirements, which limit its scalability. For Voxblox, we used the “FastTSDF” mode for RGB-D input, as the standard configuration was unstable or slow in our tests. For LiDAR data, we switched to the “MergedTSDF” configuration to ensure compatibility with sparser point clouds. These settings were chosen to reflect best-case performance for each method in its intended domain. To ensure a fair comparison, voxel sizes were adapted to sensor resolution: 0.01 m for dense RGB-D input and 0.20 m for sparse LiDAR input. This allows each method to operate under conditions that match the characteristics of its input, and helps isolate the effect of voxel structure and memory management. Tab. 1 and Tab. 2 present the quantitative results on Scannet with NC, as well as Replica combined with Oxford-Spires, respectively. Complementary qualitative examples are provided in Fig. 9 and Fig. 10. These visuals illustrate how our multi-resolution approach maintains surface detail in geometrically complex regions and avoids over-allocation in homogeneous areas, supporting both high accuracy and efficiency across sensor types.

4.3 Memory

We report memory consumption indirectly by measuring the number of mesh vertices and faces produced during reconstruction, which closely correlates with storage requirements and runtime

		0000	0010	0059	0106	0109	0181	0207	quad	math	cloi	Avg RGB-D	Avg LiDAR	Avg
PIN-SLAM	RGB-D								LiDAR					
	Acc.[cm]↓	2.680	4.069	5.789	6.540	2.802	4.161	4.004	9.942	10.271	9.082	4.292	9.765	5.934
	Comp.[cm]↓	1.078	1.060	1.464	1.732	0.862	1.648	1.085	13.764	13.960	14.584	1.276	14.103	5.124
	C-L1↓	1.879	2.565	3.626	4.136	1.832	2.905	2.544	11.853	12.116	11.833	2.784	11.934	5.529
	F-score[%]↑	97.966	94.486	87.775	83.363	96.209	93.494	93.496	83.724	83.596	82.431	92.398	83.250	89.654
VDBFusion	Acc.[cm]↓	2.217	2.473	4.615	5.472	1.603	4.090	2.874	6.511	8.669	6.511	3.336	7.230	4.504
	Comp.[cm]↓	0.925	0.707	1.122	0.885	0.516	0.928	0.779	11.645	12.684	12.657	0.837	12.329	4.285
	C-L1↓	1.571	1.590	2.869	3.179	1.059	2.509	1.826	9.078	10.677	9.584	2.086	9.780	4.394
	F-score[%]↑	97.052	95.687	88.896	85.161	97.659	89.723	94.692	89.532	87.157	87.963	92.696	88.217	91.352
Voxblox	Acc.[cm]↓	2.120	2.718	4.156	4.137	2.583	4.246	2.678	9.824	8.619	9.284	3.234	9.242	5.037
	Comp.[cm]↓	1.759	6.168	2.145	2.304	9.045	2.638	3.817	17.083	14.756	26.207	3.982	19.349	8.592
	C-L1↓	1.939	4.443	3.150	3.220	5.814	3.442	3.247	13.453	11.688	17.746	3.608	14.296	6.814
	F-score[%]↑	96.076	89.194	89.313	88.593	84.653	91.204	91.329	79.400	83.699	63.518	90.052	75.539	85.698
Supereight2†	Acc.[cm]↓	2.151	2.364	5.354	4.781	2.064	4.221	3.151	fail	fail	fail	3.441	-	-
	Comp.[cm]↓	1.457	1.294	2.002	1.484	1.183	1.983	1.470	fail	fail	fail	1.553	-	-
	C-L1↓	1.804	1.829	3.678	3.132	1.623	3.102	2.310	fail	fail	fail	2.497	-	-
	F-score[%]↑	97.478	97.399	87.040	88.131	97.073	91.905	95.252	fail	fail	fail	93.468	-	-
Supereight2‡	Acc.[cm]↓	1.955	2.364	5.250	4.176	2.064	3.834	3.096	fail	fail	fail	3.248	-	-
	Comp.[cm]↓	1.452	1.296	2.011	1.464	1.182	1.954	1.476	fail	fail	fail	1.548	-	-
	C-L1↓	1.704	1.830	3.631	2.820	1.623	2.894	2.286	fail	fail	fail	2.398	-	-
	F-score[%]↑	98.168	97.394	87.454	90.752	97.075	93.458	95.445	fail	fail	fail	94.250	-	-
N ³ -Mapping	Acc.[cm]↓	fail	1.705	2.338	3.374	1.706	2.134	1.786	fail	fail	fail	2.174	-	-
	Comp.[cm]↓	fail	1.335	1.664	1.390	1.133	2.616	1.222	fail	fail	fail	1.560	-	-
	C-L1↓	fail	1.520	2.001	2.382	1.420	2.375	1.504	fail	fail	fail	1.867	-	-
	F-score[%]↑	fail	98.811	96.398	93.671	98.076	97.034	97.293	fail	fail	fail	96.880	-	-
Ours	Acc.[cm]↓	0.963	1.587	2.981	2.822	1.364	2.789	1.701	7.134	9.061	7.015	2.030	7.737	3.742
	Comp.[cm]↓	1.053	1.079	1.414	1.202	0.961	1.968	1.234	11.898	13.116	14.208	1.273	13.074	4.813
	C-L1↓	1.008	1.333	2.198	2.012	1.162	2.379	1.366	9.516	11.088	10.612	1.637	10.405	4.267
	F-score[%]↑	99.674	98.408	94.124	94.200	98.079	96.183	97.246	89.503	85.811	84.815	96.845	86.710	93.804

Table 1. **3D Reconstruction Results.** The RGB-D data are sequences of ScanNet [Dai et al. 2017a] and LiDAR data are sequences of NC [Zhang et al. 2021]. All the pipelines are run with ground-truth fixed poses. Voxel size is set to 1 cm for RGB-D data and 20 cm for LiDAR data, and the F-score is computed with a 10 cm error threshold for the RGB-D and 20 cm for the LiDAR. Best results are highlighted as **first** and **second**. Supereight† represents single resolution grid, Supereight‡ represents multi-resolution grid. Both Supereight2† and Supereight2‡ failed on all LiDAR sequences. N³-Mapping fails on one RGB-D sequence and all the LiDAR sequences; for completeness, we report the average for RGB-D despite is calculated only on its working sequences.

memory usage. Tab. 4 summarizes average values across both RGB-D and LiDAR datasets. Our method, particularly in its multi-resolution configuration, produces significantly more compact reconstructions compared to existing approaches. In the RGB-D setup, we observe memory savings ranging from 2.0× to 7.5×. For LiDAR data, reductions range from 2.3× to 2.9× compared to other baselines, except for VDBFusion, whose mesh size is 0.7x of ours. On average, our multi-resolution compression method improves from 1.95x to 4.07x. This demonstrates the effectiveness of our variance-driven voxel allocation, which allows the system to retain high resolution in geometrically complex areas while using coarser voxels in flat or redundant regions. These gains are made possible by our design, which avoids over-allocating memory in uniform areas through adaptive resolution.

4.4 Runtimes

Tab. 4 reports both the processing time per frame (in milliseconds) and the corresponding Frame Per Seconds (FPS). Our approach outperforms all baselines in nearly every scenario. The single-resolution variant already yields significant speedups, up to 13× faster than prior state-of-the-art methods such as VDBFusion and Voxblox. The multi-resolution configuration achieves additional performance gains, especially in large-scale or sparse scenes, while preserving reconstruction quality. Notably, our method maintains real-time or near-real-time performance across all datasets, including challenging LiDAR-based sequences from NC. Competing methods such as N³-Mapping failed to complete these sequences, highlighting the scalability and robustness of our approach. These improvements

		office0	office1	office2	office3	office4	room0	room1	room2	bod	keb	obs	Avg RGB-D	Avg LiDAR	Avg
		RGB-D								LiDAR					
PIN-SLAM	Acc.[cm]↓	0.946	0.821	0.885	1.041	0.871	0.861	0.851	0.872	8.672	7.681	9.365	0.893	8.573	4.733
	Comp.[cm]↓	4.383	4.641	3.332	2.686	2.834	3.147	4.843	2.560	13.846	10.579	12.577	3.553	12.334	7.944
	C-L1↓	2.664	2.731	2.109	1.864	1.853	2.004	2.847	1.716	11.259	9.130	10.971	2.223	10.453	6.338
	F-score[%]↑	93.815	93.020	94.604	95.370	95.290	94.701	91.820	96.121	84.864	89.193	85.378	94.343	86.478	90.411
VDBFusion	Acc.[cm]↓	0.545	0.512	0.554	0.575	0.568	0.574	0.535	0.546	5.447	5.145	6.951	0.551	5.848	3.200
	Comp.[cm]↓	2.784	3.717	3.498	2.693	2.892	2.539	2.381	2.647	21.987	10.703	12.125	2.894	14.939	8.917
	C-L1↓	1.664	2.115	2.026	1.634	1.730	1.557	1.458	1.597	13.717	7.924	9.538	1.722	10.393	6.058
	F-score[%]↑	95.828	93.804	94.115	94.916	94.853	95.685	95.773	95.645	74.414	90.519	86.945	95.077	83.959	89.518
Voxblox	Acc.[cm]↓	0.577	0.682	0.699	0.896	0.597	0.622	0.746	0.574	8.484	8.772	9.001	0.674	8.753	4.714
	Comp.[cm]↓	3.331	7.529	3.452	2.434	2.863	2.184	3.116	2.826	21.861	23.340	18.994	3.467	21.398	12.433
	C-L1↓	1.954	4.106	2.075	1.665	1.730	1.403	1.931	1.700	15.172	16.056	13.998	2.070	15.076	8.573
	F-score[%]↑	95.262	87.167	94.251	95.706	94.817	96.223	94.647	94.997	73.172	70.284	76.241	94.134	73.233	83.684
Supereight2†	Acc.[cm]↓	0.819	0.790	0.985	0.952	1.211	1.109	0.932	1.323	fail	fail	fail	1.015	–	–
	Comp.[cm]↓	2.907	3.845	3.728	2.751	3.143	2.624	2.807	2.712	fail	fail	fail	3.065	–	–
	C-L1↓	1.863	2.317	2.356	1.851	2.177	1.867	1.869	2.018	fail	fail	fail	2.040	–	–
	F-score[%]↑	96.199	93.998	94.392	95.594	95.044	96.321	95.777	95.015	fail	fail	fail	95.293	–	–
Supereight2‡	Acc.[cm]↓	0.819	0.790	1.004	0.977	1.216	1.113	0.934	1.323	fail	fail	fail	1.022	–	–
	Comp.[cm]↓	2.908	3.844	3.743	2.761	3.147	2.615	2.810	2.714	fail	fail	fail	3.068	–	–
	C-L1↓	1.863	2.317	2.374	1.869	2.182	1.864	1.872	2.018	fail	fail	fail	2.045	–	–
	F-score[%]↑	96.197	93.998	94.384	95.586	95.039	96.326	95.775	95.017	fail	fail	fail	95.290	–	–
N ³ -Mapping	Acc.[cm]↓	fail	0.739	fail	fail	fail	fail	fail	fail	5.737	5.617	6.649	0.739	6.001	–
	Comp.[cm]↓	fail	3.950	fail	fail	fail	fail	fail	fail	25.728	13.379	12.389	3.950	17.165	–
	C-L1↓	fail	2.344	fail	fail	fail	fail	fail	fail	15.733	9.498	9.519	2.344	11.583	–
	F-score[%]↑	fail	96.560	fail	fail	fail	fail	fail	fail	68.460	88.026	88.458	96.560	81.648	–
NKSR	Acc.[cm]↓	fail	fail	fail	fail	fail	fail	fail	fail	8.782	7.654	8.798	–	8.411	–
	Comp.[cm]↓	fail	fail	fail	fail	fail	fail	fail	fail	15.804	11.318	11.915	–	13.013	–
	C-L1↓	fail	fail	fail	fail	fail	fail	fail	fail	12.293	9.486	10.357	–	10.712	–
	F-score[%]↑	fail	fail	fail	fail	fail	fail	fail	fail	81.660	88.295	86.650	–	85.535	–
Ours	Acc.[cm]↓	0.579	0.542	0.601	0.624	0.624	0.607	0.565	0.594	6.293	4.892	7.639	0.592	6.275	3.434
	Comp.[cm]↓	2.714	3.685	3.414	2.582	2.880	2.452	2.311	2.704	21.269	12.698	12.743	2.843	15.570	9.207
	C-L1↓	1.646	2.113	2.007	1.603	1.752	1.530	1.438	1.649	13.781	8.795	10.191	1.717	10.922	6.320
	F-score[%]↑	96.062	93.867	94.221	95.280	94.889	96.029	96.012	95.601	75.065	88.462	85.865	95.245	83.131	89.188

Table 2. **3D Reconstruction Results.** The RGB-D data are sequences of ScanNet [Straub et al. 2019] and LiDAR data are sequences of Oxford Spires Dataset [Tao et al. 2025]. All the pipelines are run with ground-truth fixed poses. Voxel size is set to 1 cm for RGB-D data and 20 cm for LiDAR data, and the F-score is computed with a 10 cm error threshold for the RGB-D and 20 cm for the LiDAR. Best results are highlighted as **first** and **second**. Supereight† represents single resolution grid, Supereight‡ represents multi-resolution grid. Both Supereight approaches run on Replica sequences but fail on Oxford Spires. N³-Mapping fails on most Replica sequences but runs on Oxford. NKSR fails on all Replica sequences but runs on Oxford Spires.

stem from three key design choices: (1) a flat hash table that enables constant-time access without hierarchical traversal overhead, (2) a fully GPU-based pipeline that avoids costly CPU-GPU data transfers, and (3) an adaptive voxel resolution strategy that reduces unnecessary computation and memory usage in low-detail regions. Together, these contribute to the significant runtime advantage of our system.

4.5 Rendering

To evaluate the rendering performance of our pipeline, we integrate our voxel grid with a Gaussian Splatting (GS) renderer and compare against GSFusion [Wei and Leutenegger 2024], a recent method designed for efficient point-based rendering. Tab. 5 reports standard perceptual metrics, PSNR, SSIM, and LPIPS, as well as rendering speed in FPS. Our multi-resolution configuration achieves

Method	RGB-D			LiDAR			Avg.		
	# Vertices ↓	# Faces ↓	Memory [MB] ↓	# Vertices ↓	# Faces ↓	Memory [MB] ↓	# Vertices ↓	# Faces ↓	Memory [MB] ↓
VDBFusion	4576631	8575618	331.2	321179	587890	15.4	2448905	4581754	173.3
PIN-SLAM	9160908	16851839	658.8	896877	1626551	64.2	5028893	9239195	361.5
Voxblox	5841313	7168535	373.6	828294	845420	50.7	3334804	4006978	212.15
Supereight2 [†]	27171273	9057091	1163.4	–	–	–	–	–	–
Supereight2 [‡]	19247955	6415985	817.0	–	–	–	–	–	–
Ours (single)	4501002	4070646	242.2	753320	1246638	22.8	2431648	2349585	132.5
Ours (multi)	2068369	3752706	155.7	615204	1035968	21.60	1206970	2173218	88.65

Table 3. **Average memory usage.** We report the number of vertices and faces as a proxy for memory consumption, as they correlate with mesh complexity and storage requirements. Best values are marked as **first** and **second**. Dash “–” indicates missing values due to failure. Supereight[†] represents resolution grid, Supereight[‡] represents multi-resolution grid. Our multi-resolution setup improves from 2.0x to 7.5x in the RGB-D setup and from 2.3x to 2.9x in the LiDAR setup (excluding VDBFusion meshes that are 0.7x compared to ours). On average, the improvement is from 1.95x to 4.07.

Method	ScanNet		Replica		Oxford-Spires		NC	
	Time [ms] ↓	FPS ↑	Time [ms] ↓	FPS ↑	Time [ms] ↓	FPS ↑	Time [ms] ↓	FPS ↑
VDBFusion	17.45	5.71	519	1.93	40.31	24.65	103.62	9.61
PIN-SLAM	67.15	14.80	68	14.7	75.99	13.15	113.68	8.79
N ³ -Mapping	184.67	5.40	–	–	255.44	3.91	–	–
NKSR	–	–	–	–	16.05	62.28	–	–
Voxblox	132.06	7.57	373	2.68	61.72	16.17	162.47	6.14
Supereight2 [†]	73.94	13.53	100.87	9.92	–	–	–	–
Supereight2 [‡]	79.52	12.55	98.11	10.20	–	–	–	–
Ours (single)	15.11	64.30	20.45	48.9	14.27	68.86	30.18	21.6
Ours (multi)	16.79	59.34	37.50	26	16.39	61	28.17	35.25

Table 4. **Runtime comparison.** Time [ms] refers to the total computation time for one frame of the sequence, while FPS refers to processing speed. Best results are highlighted as **first** and **second**. Dash “–” indicates missing values due to failure. Supereight[†] represents single resolution grid, Supereight[‡] represents multi-resolution grid. Our runtimes show an improvement from 2-3x to 13x compared to existing SOTA.

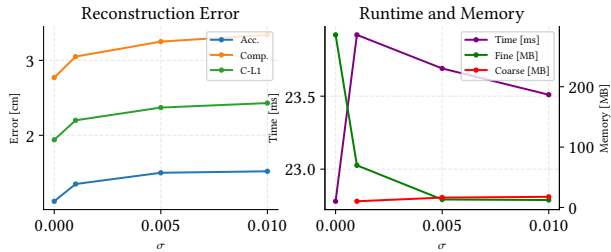


Fig. 7. **Comparison of reconstruction error, runtimes, and memory usage for different variance thresholds σ .** The left plot reports accuracy, completeness, and Chamfer distance, while the right plot shows runtime and memory consumption of the fine- and coarse-level voxel structures. The results confirm that low thresholds yield the highest accuracy at the cost of increased fine-resolution memory, while moderate thresholds preserve comparable quality with reduced memory usage.

consistently better results across all quality metrics. This improvement is attributed to our adaptive voxel grid, which maintains high-resolution detail in complex regions while avoiding redundant splat

Method	PSNR ↑	SSIM ↑	LPIPS ↓	FPS ↑	Time[ms] ↓
GSFusion	34.65	0.949	0.056	14.99	66.67
Ours	33.90	0.949	0.057	28.05	35.65
Ours (+iterations)	34.27	0.951	0.052	14.22	70.32
Ours (multi)	35.73	0.960	0.044	23.70	42.19

Table 5. **Rendering quality and performance.** Comparison of perceptual metrics (PSNR, SSIM, LPIPS) and rendering speed (FPS) on Replica dataset [Straub et al. 2019]. Best results are highlighted as **first** and **second**. Ours(+iterations) means we set more iterations to optimize GS parameters, using the single-resolution grid. The multi-resolution setup improves overall rendering quality in the GS pipeline, with a small computational overhead. Nevertheless, our GPU-based quad-tree implementation achieves better overall performance compared to other GSFusion.

Method	Mesh [MB] ↓	Acc.[cm] ↓	Comp.[cm] ↓	C-L1 ↓	F-Score[%] ↑
GSFusion	454.025	0.608	2.777	1.692	95.192
Ours	132.650	0.614	2.832	1.723	95.106

Table 6. **Reconstruction quality and performance.** We compare the two approaches on the same sequences of Tab. 5 i.e., Replica dataset [Straub et al. 2019]. Best results are highlighted as **first** and **second**. Considering the synthetic nature of the data, the results are almost equal at mm level. Nevertheless, the mesh size of GSFusion is almost 3x times bigger than ours.

generation in homogeneous areas. This leads to more accurate reconstructions and better perceptual fidelity in rendered images. In terms of performance, our base version achieves the highest FPS (28.05), demonstrating the strength of our fully GPU-based quad-tree rendering mechanism. Unlike prior methods that rely on fixed splat densities or CPU-side pre-processing, our implementation leverages a parallel quad-tree structure to dynamically manage splat density at runtime. This enables both high-quality and real-time rendering, at the expense of efficiency and visual quality. In Tab. 6, we provide a comparison of reconstruction results with GSFusion, noting that it uses Supereight2 [Funk et al. 2021] as the underlying mapping pipeline. In Fig. 11 and Fig. 12, we present qualitative examples

		0000	0010	0059	0106	0109	0181	0207	quad	math	cloi	Avg RGB-D	Avg LiDAR	Avg
Ours	RGB-D								LiDAR					
	Acc.[cm]↓	0.963	1.587	2.981	2.822	1.364	2.789	1.701	7.134	9.061	7.015	2.030	7.737	3.742
	Comp.[cm]↓	1.053	1.079	1.414	1.202	0.961	1.968	1.234	11.898	13.116	14.208	1.273	13.074	4.813
	C-L1↓	1.008	1.333	2.198	2.012	1.162	2.379	1.366	9.516	11.088	10.612	1.637	10.405	4.267
	F-score[%]↑	99.674	98.408	94.124	94.200	98.079	96.183	97.246	89.503	85.811	84.815	96.845	86.710	93.804
Ours (Multi)	Acc.[cm]↓	1.207	1.988	3.688	3.671	1.757	3.424	2.047	10.763	12.314	10.326	2.540	11.135	5.119
	Comp.[cm]↓	1.036	1.297	1.433	1.372	1.183	2.006	1.088	18.810	16.977	22.107	1.345	19.298	6.731
	C-L1↓	1.121	1.642	2.560	2.522	1.470	2.715	1.567	14.786	14.646	16.217	1.943	15.216	5.925
	F-score[%]↑	99.503	98.127	92.575	91.716	98.029	94.902	97.094	81.002	77.864	76.170	95.992	78.346	90.698

Table 7. **Single vs. Multi-resolution in 3D Reconstruction.** The single-resolution baseline uses a fixed voxel size of 20 cm, while our multi-resolution setup dynamically adapts voxel sizes between 20 cm and 40 cm. Performance varies across sensor modalities due to data density. For RGB-D input, the dense observations allow the multi-resolution approach to match the accuracy of the fixed-resolution baseline while reducing memory usage (Tab. 3). In contrast, for LiDAR input, sparser measurements limit the effectiveness of coarser voxels, leading to reduced reconstruction fidelity. This highlights the trade-off between adaptivity and resolution, especially in low-density regimes.

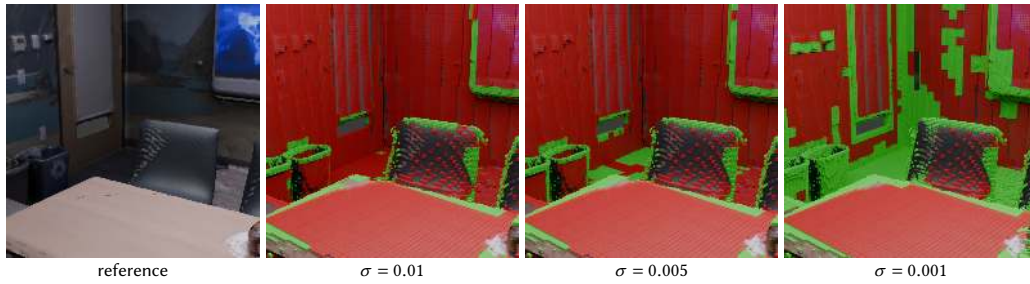


Fig. 8. **Comparison of different variance thresholds σ .** In red, the coarser voxels, in green, the finer ones. From left to right: the reference ground-truth mesh and the underlying grid at the respective thresholds. For simplicity, we show just two levels at a time. The sequence is part of the Replica dataset [Straub et al. 2019].

of NVS quality and 3D reconstruction quality, respectively. Both pipelines required approximately 5962 MB for the Gaussian representation. Additionally, our method used only 486 MB to store the map and quad-tree, accounting for just 8% of the total GPU memory needed for the entire process, demonstrating efficient resource use despite the added adaptive structures. For mapping alone, GSFusion takes 10.39 ms, while our approach takes 6.81 ms.

5 Ablation on Multi-resolution

Our approach employs a hash-based voxel grid in which spatial resolution is adaptively controlled by local TSDF variance. This enables the system to allocate finer voxels in geometrically complex or detailed regions while using coarser voxels in smoother or less informative areas, leading to substantial reductions in memory usage. To assess the effectiveness of this adaptive-resolution strategy, we compare our method with a single-resolution baseline in Tab. 7. The baseline employs a uniform voxel size of 1 cm for RGB-D data and 20 cm for LiDAR data. In contrast, our multi-resolution configuration adapts voxel sizes between 1 cm (finest) and 2 cm (coarsest) for RGB-D, and between 20 cm (finest) and 40 cm (coarsest) for LiDAR. As expected, the single-resolution setup achieves slightly higher reconstruction accuracy due to uniformly fine sampling, serving as an

upper bound. However, our multi-resolution variant achieves nearly equivalent reconstruction quality with significantly fewer mesh vertices and faces, translating into meaningful memory savings. This illustrates a key trade-off between geometric precision and memory efficiency, which is particularly advantageous in large-scale or resource-constrained environments.

Interestingly, in the rendering setup Tab. 5, the multi-resolution configuration outperforms even the fixed-resolution baseline in perceptual quality, because the adaptive voxel structure spawns more splats in detailed regions and fewer in flat areas. Consequently, we achieve sharper renderings and improved metrics while maintaining high frame rates. These benefits are further enhanced by our GPU-accelerated quad-tree, which enables efficient real-time splat management. Overall, these results confirm our variance-driven resolution control as a flexible and effective strategy for balancing accuracy and memory. The approach proves well-suited for a range of real-time applications, including dense 3D reconstruction and photorealistic rendering, where performance and resource constraints are critical.

5.1 Variance Thresholds

Since our adaptive resolution scheme depends directly on the local TSDF variance, the choice of the variance threshold σ is critical in balancing between reconstruction accuracy and memory usage. Lower thresholds favor the allocation of finer voxels over larger areas, improving geometric precision at the cost of increased memory. Conversely, higher thresholds limit fine-resolution allocation to regions of steep TSDF variation, reducing resource usage but potentially omitting small-scale details. To investigate this trade-off, we evaluate different values of the variance threshold σ on the Replica dataset [Straub et al. 2019]. Fig. 7 reports their impacts on reconstruction quality metrics, while Fig. 8 qualitatively illustrates how varying the threshold changes the distribution of coarse and fine voxels in the scene. The results confirm the expectation: very low thresholds yield the most accurate reconstructions but allocate significantly more fine-resolution voxels, whereas moderate thresholds keep the reconstruction quality comparable, with reduced memory usage.

6 Conclusion

We presented a novel multi-resolution voxel grid framework for real-time 3D reconstruction that adapts resolution based on the local variance of TSDF observations. Unlike traditional octree or VDB-based methods, our approach leverages a flat hash table to manage voxel blocks of varying resolution, enabling constant-time access and full GPU parallelization. This design supports efficient integration and rendering while remaining sensor-agnostic, handling both RGB-D and LiDAR inputs. Our experiments demonstrate that the proposed method achieves competitive reconstruction quality, with up to $4\times$ memory savings and $13\times$ speedup compared to fixed-resolution baselines. The integration with a GPU-accelerated quad-tree further enables high-performance rendering through Gaussian Splatting, balancing visual fidelity and efficiency. By combining adaptive resolution, scalability, and real-time performance, our method offers a practical and extensible solution for large-scale 3D mapping in graphics and robotics applications. We release our implementation as open-source to encourage further research and adoption.

Acknowledgments

This work has been partially supported by PNRR MUR project PE0000013-FAIR

References

- John Amanatides and Andrew Woo. 1987. A Fast Voxel Traversal Algorithm for Ray Tracing. In *Proc. of Eurographics (EG)*. doi:10.2312/egtp.19871000
- Leonardo Brizi, Emanuele Giacomini, Luca Di Giammarino, Simone Ferrari, Omar Salem, Lorenzo De Reboti, and Giorgio Grisetti. 2024. VBR: A Vision Benchmark in Rome. In *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. 15868–15874. doi:10.1109/ICRA57147.2024.10611395
- Brian Curless and Marc Levoy. 1996. A volumetric method for building complex models from range images. In *Proc. of the ACM SIGGRAPH Conf. on Computer Graphics and Interactive Techniques*. 303–312. doi:10.1145/237170.237269
- Krzysztof Cwian, Luca Di Giammarino, Simone Ferrari, Thomas Ciarfuglia, Giorgio Grisetti, and Piotr Skrzypczyński. 2025. MAD-BA: 3D LiDAR Bundle Adjustment – From Uncertainty Modelling to Structure Optimization. *IEEE Robotics and Automation Letters (RA-L)* 10, 7 (July 2025), 7254–7261. doi:10.1109/LRA.2025.3573628
- Angela Dai, Angel X. Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. 2017a. ScanNet: Richly-Annotated 3D Reconstructions of Indoor Scenes. In *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. 2432–2443. doi:10.1109/CVPR.2017.261
- Angela Dai, Matthias Nießner, Michael Zollhöfer, Shahram Izadi, and Christian Theobalt. 2017b. BundleFusion: Real-Time Globally Consistent 3D Reconstruction Using On-the-Fly Surface Reintegration. *ACM Trans. on Graphics* 36, 3 (June 2017), 1–18. doi:10.1145/3054739
- Luca Di Giammarino, Leonardo Brizi, Tiziano Guadagnino, Cyrill Stachniss, and Giorgio Grisetti. 2022. MD-SLAM: Multi-cue Direct SLAM. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*. 11047–11054. doi:10.1109/IROS47612.2022.9981147
- Luca Di Giammarino, Emanuele Giacomini, Leonardo Brizi, Omar Salem, and Giorgio Grisetti. 2023. Photometric LiDAR and RGB-D Bundle Adjustment. *IEEE Robotics and Automation Letters (RA-L)* 8, 7 (July 2023), 4362–4369. doi:10.1109/LRA.2023.3281907
- Simone Ferrari, Luca Di Giammarino, Leonardo Brizi, and Giorgio Grisetti. 2024. MAD-ICP: It Is All About Matching Data – Robust and Informed LiDAR Odometry. *IEEE Robotics and Automation Letters (RA-L)* 9, 11 (Nov. 2024), 9175–9182. doi:10.1109/LRA.2024.3456509
- Nils Funk, Juan Tarrio, Sotiris Papatheodorou, Marija Popović, Pablo F. Alcantarilla, and Stefan Leutenegger. 2021. Multi-Resolution 3D Mapping With Explicit Free Space Representation for Fast and Accurate Mobile Robot Motion Planning. *IEEE Robotics and Automation Letters (RA-L)* 6, 2 (April 2021), 3553–3560. doi:10.1109/LRA.2021.3061989
- Emanuele Giacomini, Luca Di Giammarino, Lorenzo De Reboti, Giorgio Grisetti, and Martin R. Oswald. 2025. Splat-LOAM: Gaussian Splatting LiDAR Odometry and Mapping. In *Proc. of the IEEE Intl. Conf. on Computer Vision (ICCV)*. 27630–27639.
- Jiahui Huang, Zan Gojic, Matan Atzmon, Or Litany, Sanja Fidler, and Francis Williams. 2023. Neural Kernel Surface Reconstruction. In *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. 4369–4379. doi:10.1109/CVPR52729.2023.00425
- Nikhil Keetha, Jay Karhade, Krishna Murthy Jatavallabhula, Gengshan Yang, Sebastian Scherer, Deva Ramanan, and Jonathon Luiten. 2024. SplatTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM. In *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. 21357–21366. doi:10.1109/CVPR52733.2024.02018
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. on Graphics* 42, 4 (2023), 139–1.
- Eric Lengyel. 2010. Transition Cells for Dynamic Multiresolution Marching Cubes. *Journal of Graphics, GPU, and Game Tools (JGGT)* 15, 2 (May 2010), 99–122. doi:10.1080/2151237X.2011.563682
- William E. Lorensen and Harvey E. Cline. 1987. Marching cubes: A high resolution 3D surface construction algorithm. In *Proc. of the ACM SIGGRAPH Conf. on Computer Graphics and Interactive Techniques*. 163–169. doi:10.1145/37401.37422
- Hidehiko Matsuki, Riku Murai, Paul H. J. Kelly, and Andrew J. Davison. 2024. Gaussian Splatting SLAM. In *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. 18039–18048. doi:10.1109/CVPR52733.2024.01708
- Lars Mescheder, Michael Oechsle, Michael Niemeyer, Sebastian Nowozin, and Andreas Geiger. 2019. Occupancy Networks: Learning 3D Reconstruction in Function Space. In *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. 4455–4465. doi:10.1109/CVPR.2019.00459
- Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. 2022. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Trans. on Graphics* 41, 4 (2022), 1–15.
- Ken Museth. 2013. VDB: High-resolution sparse volumes with dynamic topology. *ACM Trans. on Graphics* 32, 3 (2013), 1–22.
- Ken Museth. 2021. NanoVDB: A GPU-Friendly and Portable VDB Data Structure For Real-Time Rendering And Simulation. In *ACM SIGGRAPH Talks*. 1–2. doi:10.1145/3450623.3464653
- Richard A. Newcombe, Shahram Izadi, Otmar Hilliges, David Molyneaux, David Kim, Andrew J. Davison, Pushmeet Kohi, Jamie Shotton, Steve Hodges, and Andrew Fitzgibbon. 2011. KinectFusion: Real-time Dense Surface Mapping and Tracking. In *Proc. of the Intl. Symposium on Mixed and Augmented Reality (ISMAR)*. 127–136. doi:10.1109/ISMAR.2011.6092378
- Matthias Nießner, Michael Zollhöfer, Shahram Izadi, and Marc Stamminger. 2013. Real-time 3D reconstruction at scale using voxel hashing. *ACM Trans. on Graphics* 32, 6 (2013), 1–11.
- Helen Oleynikova, Zachary Taylor, Marius Fehr, Roland Siegwart, and Juan Nieto. 2017. Voxblox: Incremental 3d euclidean signed distance fields for on-board mav planning. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*. IEEE, 1366–1373.
- Yue Pan, Xingguang Zhong, Louis Wiesmann, Thorbjörn Posewsky, Jens Behley, and Cyrill Stachniss. 2024. PIN-SLAM: LiDAR SLAM Using a Point-Based Implicit Neural Representation for Achieving Global Map Consistency. *IEEE Trans. on Robotics (TRO)* 40 (2024), 4045–4064. doi:10.1109/TRO.2024.3422055
- Zhexi Peng, Tianjia Shao, Yong Liu, Jingke Zhou, Yin Yang, Jingdong Wang, and Kun Zhou. 2024. RTG-SLAM: Real-time 3D Reconstruction at Scale Using Gaussian Splatting. In *Proc. of the ACM SIGGRAPH Conf. on Computer Graphics and Interactive Techniques*. 1–11. doi:10.1145/3641519.3657455

- Thomas Schöps, Torsten Sattler, and Marc Pollefeys. 2019. BAD SLAM: Bundle Adjusted Direct RGB-D SLAM. In *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. 134–144. doi:10.1109/CVPR.2019.00022
- Shuangfu Song, Junqiao Zhao, Kai Huang, Jiaye Lin, Chen Ye, and Tiantian Feng. 2024. N³-Mapping: Normal Guided Neural Non-Projective Signed Distance Fields for Large-Scale 3D Mapping. *IEEE Robotics and Automation Letters (RA-L)* 9, 6 (June 2024), 5935–5942. doi:10.1109/LRA.2024.3396638
- Julian Straub, Thomas Whelan, Lingni Ma, Yufan Chen, Erik Wijmans, Simon Green, Jakob J. Engel, Raul Mur-Artal, Carl Ren, Shobhit Verma, Anton Clarkson, Mingfei Yan, Brian Budge, Yajie Yan, Xiaqing Pan, June Yon, Yuyang Zou, Kimberly Leon, Nigel Carter, Jesus Briales, Tyler Gillingham, Elias Mueggler, Luis Pesqueira, Manolis Savva, Dhruv Batra, Hauke M. Strasdat, Renzo De Nardi, Michael Goesele, Steven Lovegrove, and Richard Newcombe. 2019. The Replica Dataset: A Digital Replica of Indoor Spaces. doi:10.48550/arXiv.1906.05797 arXiv:1906.05797 [cs]
- Yifu Tao, Miguel Ángel Muñoz-Bañón, Lintong Zhang, Jiahao Wang, Lanke Frank Tarimo Fu, and Maurice Fallon. 2025. The Oxford Spires Dataset: Benchmarking Large-Scale LiDAR-visual Localisation, Reconstruction and Radiance Field Methods. *Intl. Journal of Robotics Research (IJRR)* (Sept. 2025), 02783649251369905. doi:10.1177/02783649251369905
- Matthias Teschner, Bruno Heidelberger, Matthias Müller, Danat Pomerantes, and Markus H. Gross. 2003. Optimized Spatial Hashing for Collision Detection of Deformable Objects. In *Proc. of the Vision, Modeling, and Visualization Workshop (VMV)*, Thomas Ertl (Ed.). 47–54.
- Emanuele Vespa, Nils Funk, Paul HJ Kelly, and Stefan Leutenegger. 2019. Adaptive-resolution octree-based volumetric SLAM. In *Proc. of the International Conference on 3D Vision (3DV)*. IEEE, 654–662.
- Emanuele Vespa, Nikolay Nikolov, Marius Grimm, Luigi Nardi, Paul H. J. Kelly, and Stefan Leutenegger. 2018. Efficient Octree-Based Volumetric SLAM Supporting Signed-Distance and Occupancy Mapping. *IEEE Robotics and Automation Letters (RA-L)* 3, 2 (April 2018), 1144–1151. doi:10.1109/LRA.2018.2792537
- Ignacio Vizzo, Tiziano Guadagnino, Jens Behley, and Cyrill Stachniss. 2022. VDBFusion: Flexible and Efficient TSDF Integration of Range Sensor Data. *Sensors* 22, 3 (Feb. 2022), 1296. doi:10.3390/s22031296
- Zhou Wang, A.C. Bovik, H.R. Sheikh, and E.P. Simoncelli. 2004. Image Quality Assessment: From Error Visibility to Structural Similarity. *IEEE Trans. on Image Processing (TIP)* 13, 4 (April 2004), 600–612. doi:10.1109/TIP.2003.819861
- Jiaxin Wei and Stefan Leutenegger. 2024. GSFusion: Online RGB-D Mapping Where Gaussian Splatting Meets TSDF Fusion. *IEEE Robotics and Automation Letters (RA-L)* 9, 12 (Dec. 2024), 11865–11872. doi:10.1109/LRA.2024.3502065
- B. P. Welford. 1962. Note on a Method for Calculating Corrected Sums of Squares and Products. *Technometrics* 4, 3 (Aug. 1962), 419–420. doi:10.1080/00401706.1962.10490022
- Francis Williams, Jiahui Huang, Jonathan Swartz, Gergely Klar, Vijay Thakkar, Matthew Cong, Xuanchi Ren, Ruilong Li, Clement Fuji-Tsang, Sanja Fidler, Eftychios Sifakis, and Ken Museth. 2024. fVDB : A Deep-Learning Framework for Sparse, Large Scale, and High Performance Spatial Intelligence. *ACM Trans. on Graphics* 43, 4 (July 2024), 133:1–133:15. doi:10.1145/3658226
- Ji Zhang and Sanjiv Singh. 2014. LOAM: Lidar Odometry and Mapping in Real-time. In *Proc. of Robotics: Science and Systems (RSS)*. doi:10.15607/RSS.2014.X.007
- Lintong Zhang, Marco Camurri, and M. Fallon. 2021. Multi-Camera LiDAR Inertial Extension to the Newer College Dataset. *ArXiv* (Dec. 2021).
- Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. 2018. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. In *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. 586–595.
- Jianhao Zheng, Daniel Barath, Marc Pollefeys, and Iro Armeni. 2024. MAP-ADAPT: Real-Time Quality-Adaptive Semantic 3D Maps. In *Proc. of the Europ. Conf. on Computer Vision (ECCV)*, Aleš Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol (Eds.). 220–237. doi:10.1007/978-3-031-72933-1_13
- Zihan Zhu, Songyou Peng, Viktor Larsson, Zhaopeng Cui, Martin R. Oswald, Andreas Geiger, and Marc Pollefeys. 2024. NICER-SLAM: Neural Implicit Scene Encoding for RGB SLAM. In *Proc. of the International Conference on 3D Vision (3DV)*. 42–52. doi:10.1109/3DV62453.2024.00096

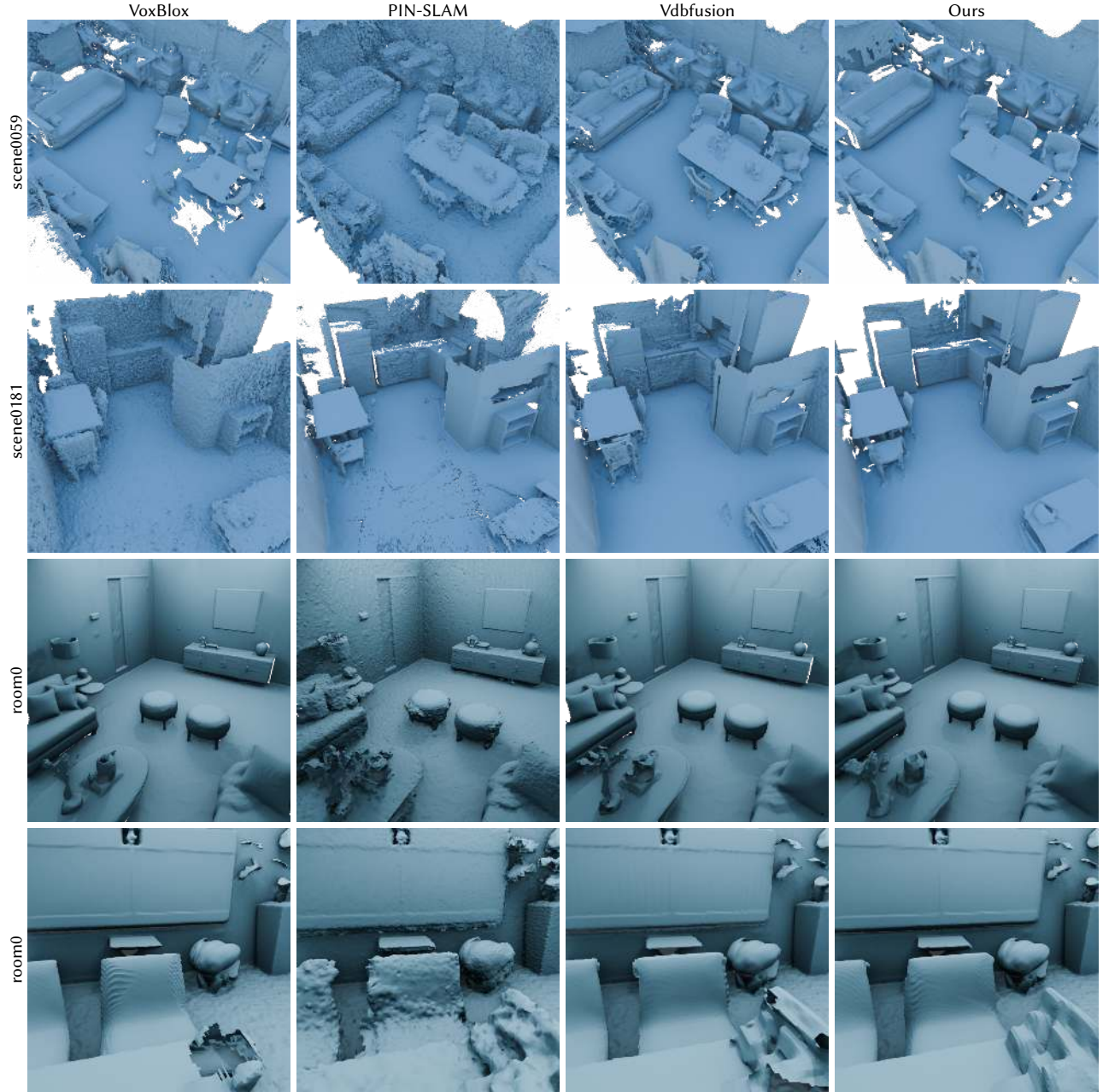


Fig. 9. **Qualitative results on RGB-D Datasets.** The images show the mapping results on scene0059 and scene0181 of ScanNet dataset [Dai et al. 2017a] and sequence room0 of Replica dataset [Straub et al. 2019]. We compare our method against other SOTA.

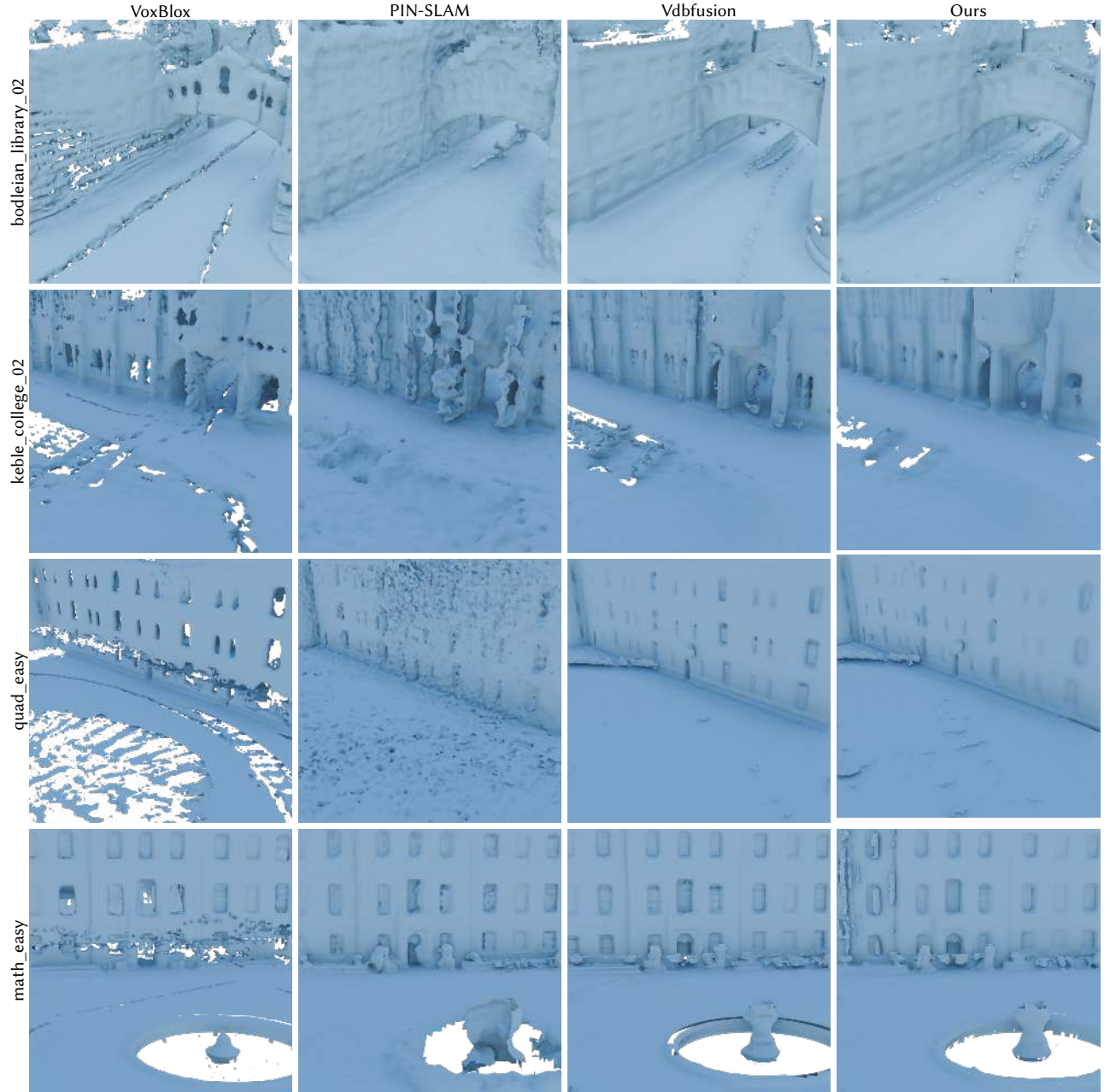


Fig. 10. **Qualitative results on LiDAR Datasets.** The images show the mapping results on sequences bodleian-library-02 and keble-college-02 Oxford-Spires [Tao et al. 2025] and sequences quad-easy and math-easy of NC [Zhang et al. 2021]. We compare our method against other SOTA.



Fig. 11. **Qualitative NVS results on RGB-D Datasets.** Column 1 and 3 shows rendered RGB frames while column 2 and 4 shows the corresponding SSIM error.

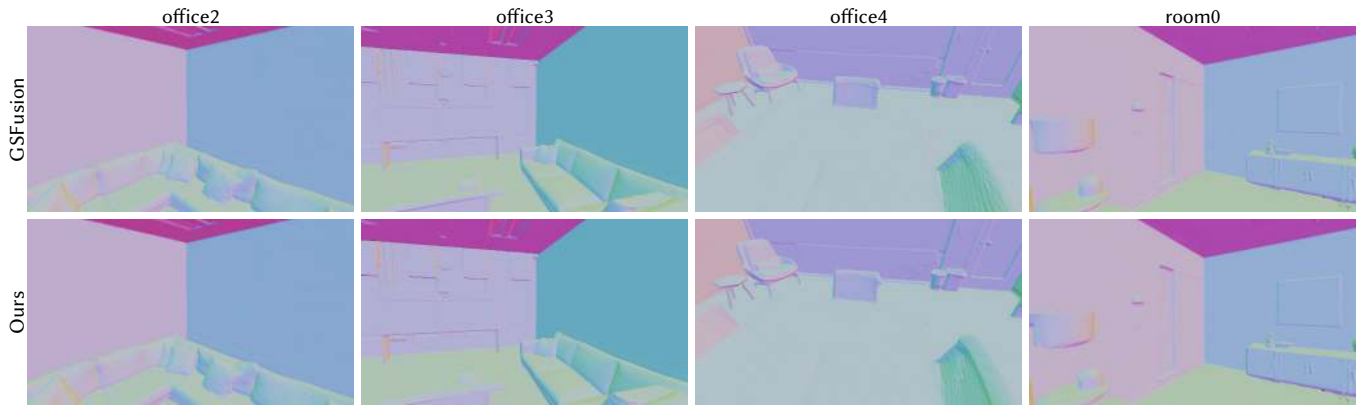


Fig. 12. **Qualitative reconstruction results on RGB-D Datasets.** The images show the mapping results on office2, office3, office4, and room0 of Replica dataset [Straub et al. 2019]. We compare our method against GSFusion[Wei and Leutenegger 2024].